
Graph Distillation: Transferring Attribution-Graph Structure from a Large Teacher to a Small Student

Eshan Singhal*
esinghal@seas.upenn.edu

Audhav Durai*
audhav@wharton.upenn.edu

Vedant Gaur*
vedantg@wharton.upenn.edu

Praneel Varshney*
pvarsh@seas.upenn.edu

Abstract

[Abstract. One paragraph: the gap between output-level distillation and mechanism transfer; graph distillation as a supernode-level structural loss on attribution graphs; the Llama-3.1-8B $\rightarrow$ Llama-3.2-1B arithmetic setting; headline result.]

1 Introduction

[Introduction. Why matching outputs need not transfer the teacher’s algorithm; why attribution graphs give a model-agnostic handle on the mechanism; contributions.]

2 Related Work

[Related work. Knowledge distillation and feature/representation matching; mechanistic interpretability and circuit discovery (attribution patching, transcoders, attribution graphs); prior attempts at mechanism-level transfer.]

3 Method

3.1 Overview

Standard knowledge distillation constrains only the *output* of the student and says nothing about how that output was produced [Hinton et al., 2015]. We add a second constraint on the *internal computation*: for a given arithmetic prompt we build an attribution graph over MLP neurons in each model, coarsen it into a small set of functionally-labelled *supernodes* that exist in both models, and require the teacher and the student to route causal influence between those supernodes in the same proportions.

Let $\mathcal{T}$ be the frozen teacher (Llama-3.1-8B) and $\mathcal{S}$ the student (Llama-3.2-1B) with trainable parameters $\theta_{\mathcal{S}}$. For a prompt x , the pipeline of Sections 3.2–3.4 produces one $K \times K$ supernode adjacency matrix per model, $W^{\mathcal{T}}(x)$ and $W^{\mathcal{S}}(x; \theta_{\mathcal{S}})$, whose row t says *where supernode t draws its input from*. Supernodes are identified by a semantic label (“ones digit of the second operand”, “magnitude of the sum”, ...) rather than by neuron index, so the rows and columns of the two matrices correspond even though the models differ in width, depth, and neuron basis.

*Equal contribution.

Dividing a row by its ℓ_1 norm turns it into a distribution over sources — supernode t ’s *routing profile*, the fraction of its incoming causal influence contributed by each other supernode. Writing P_t and Q_t for the teacher’s and the student’s routing profile for supernode t , and $\mathcal{A}$ for the supernodes present in both models, the graph loss is the mean divergence between matched profiles,

$$\mathcal{L}_{\text{graph}}(\theta_S) = \frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \text{KL}(P_t \parallel Q_t), \quad (1)$$

with P_t detached, since the teacher is fixed. The direction matters: forward KL penalises the student for failing to reproduce an edge the teacher uses, but is silent wherever the teacher puts no mass, so it asks the student to cover the teacher’s routing rather than to match it exactly.

The full objective adds (1) to the ordinary temperature-scaled KD term,

$$\mathcal{L}(\theta_S) = \underbrace{\frac{\tau^2}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \text{KL}\left(\text{softmax}(z_p^T/\tau) \parallel \text{softmax}(z_p^S/\tau)\right)}_{\mathcal{L}_{\text{KD}}} + \lambda \mathcal{L}_{\text{graph}}(\theta_S), \quad (2)$$

where z_p are logits at non-padding position p , $\tau = 2$, and $\lambda = 1$.

3.2 Attribution graphs over MLP neurons

Our graph construction follows the circuit-tracing methodology of Ameisen et al. [2025], Lindsey et al. [2025]: represent one forward pass as a directed weighted graph whose nodes are localised computational units — active at a specific layer and token position — and whose edge A_{ij} measures the *direct* linear effect of node j on node i , holding fixed everything off the direct path.

A node i is a triple (ℓ_i, p_i, n_i) : layer, token position, neuron index. It carries a *write vector* $v_i^{\text{out}} = a_i \cdot W_{\text{down}}[:, n_i]$, what the neuron adds to the residual stream, and a *read direction* $v_i^{\text{in}} = \partial a_i / \partial h$, the residual-stream direction it responds to. Edges are gradient–activation products, i.e. attribution patching [Nanda, 2023]: for a target node i we form the scalar $f_i = \langle h_{p_i}^{(\ell_i)}, \text{sg}[v_i^{\text{in}}] \rangle$ — the linearised pre-activation of neuron i , with the read direction stop-gradiented so that only the *input* to the neuron is differentiated — and set

$$A_{ij} = \left\langle \nabla_{m_{p_j}^{(\ell_j)}} f_i, v_j^{\text{out}} \right\rangle, \quad (3)$$

where $m^{(\ell_j)}$ is the output of MLP block ℓ_j . Token-embedding nodes are handled the same way with v^{out} replaced by the embedding vector, and logit nodes use $f_c = \langle h_{-1}^{(L)}, W_U[:, c] - \overline{W_U} \rangle$ with the mean-subtracted unembedding column, following Ameisen et al. [2025]. The logit targets c are the smallest set of top tokens whose probabilities under $\text{softmax}(z/\tau)$, $\tau = 2$, sum to 0.95, capped at ten. Since $n_{\text{pos}} \times d_{\text{mlp}}$ candidate neurons per layer is far too many, we keep the top 10% per layer by write-vector norm; activations at the BOS position are zeroed so the attention sink is never selected.

Why raw neurons are enough. The published pipeline does not attribute to neurons: it first replaces every MLP with a cross-layer transcoder, so that nodes are sparse, more nearly monosemantic features [Dunefsky et al., 2024]. We attribute directly to raw MLP neurons, and we think this is sound for three reasons. First, the construction in (3) requires only that a node’s contribution to the residual stream be linear in its activation. MLP neurons satisfy this exactly, $v_i^{\text{out}} = a_i W_{\text{down}}[:, n_i]$, so the linearisation is no less valid over neurons than over transcoder features. Second, the transcoder is introduced for interpretability, not for the validity of the attribution, and it is not free: because it only approximately reconstructs the MLP output, the replacement model must carry *error nodes* for the reconstruction residual, which absorb attribution and admit no interpretation of their own. Attributing to raw neurons leaves no such residual — the graph is over the computation the model actually performs. Third, the real cost is polysemanticity: a single neuron rarely corresponds to one human-legible concept. But this is exactly what the supernode step repairs, and it is worth noting that the individual feature is not the interpretable object in the published work either — features there are grouped by hand into supernodes before any circuit is read off. We automate that grouping (Section 3.3), so in both cases the interpretable unit is the group, and in ours the loss (1) is defined over groups and never over individual nodes.

3.3 Grouping neurons into supernodes

Raw neuron graphs are far too large and too model-specific to compare across a 1B and an 8B model. Where supernodes in the published work are drawn by hand from feature visualisations, we replace the manual step with an automatic, purely functional criterion, which is what makes the grouping reproducible across two different models.

Every neuron is evaluated on a fixed probe set: all two-digit addition prompts “ $a+b=$ ” for $a, b \in \{0, \dots, 99\}$, i.e. all 10^4 points of the 100×100 grid. This yields for neuron i an *activation grid* $T_i \in \mathbb{R}^{100 \times 100}$, with $T_i[a, b]$ its activation on that prompt — a direct empirical description of what the neuron responds to, as a picture. The node panels of Figure 1 are examples.

We then ask which of a fixed set of arithmetically meaningful partitions of the grid best explains that picture. Each candidate is a binary mask $M \in \{0, 1\}^{100 \times 100}$; for a prompt with operands (a^*, b^*) and sum $s^* = a^* + b^*$, the categories are

category	mask $M[a, b] = 1$ iff	category	mask $M[a, b] = 1$ iff
arg1 range	$a = a^*$	carry	$(a \bmod 10) + (b \bmod 10) \geq 10$
arg1 units	$a \equiv a^* \pmod{10}$	sum range	$a + b = s^*$
arg2 range	$b = b^*$	sum units	$a + b \equiv s^* \pmod{10}$
arg2 units	$b \equiv b^* \pmod{10}$	arg1 units and arg2 units	composite (see below)

The score of mask M for neuron i is the fraction of the variance of T_i that the mask explains. For a binary regressor this is the squared Pearson correlation, and is exactly the one-way ANOVA η^2 between the two groups the mask induces:

$$s_{i,M} = \frac{(\langle T_i - \bar{T}_i, M - \bar{M} \rangle)^2}{\|T_i - \bar{T}_i\|_2^2 \cdot \|M - \bar{M}\|_2^2} \in [0, 1], \quad (4)$$

and a category’s score s_i^c is the best score over its masks. The composite category `arg1 units` and `arg2 units` instead scores $\min(s_i^{\text{arg1 units}}, s_i^{\text{arg2 units}})$, so it fires only for a neuron sensitive to both ones digits at once. Neurons are ranked not by s_i^c but by *specificity*, the margin over the best competing category, $\delta_i^c = s_i^c - \max_{c' \neq c} s_i^{c'}$, and supernode S_c is the top $n_{\text{per}} = 10$ neurons by δ_i^c . Ranking by margin is what keeps the supernodes functionally disjoint: a neuron firing on a broad band of a scores well on several correlated categories at once, and only the margin separates a genuine ones-digit detector from a magnitude detector that happens to correlate with one.

Two categories are handled differently, because their neurons are defined by what they *write* rather than by what they read. For `sum range` and `sum units` we rank candidates by how closely the neuron’s direct logit attribution $\text{softmax}(v_i^{\text{out}^\top} W_U)$, restricted to the number tokens 0–200, matches the target distribution in KL — a point mass at s^* for `sum range`, and uniform over $\{n : n \equiv s^* \pmod{10}\}$ for `sum units`. The same score defines two further `dla` supernodes, one per sum target, collecting the neurons that write those distributions to the output most directly; these are the bottom two nodes on each side of Figure 1.

Because the same category list produces the same supernode labels in both models, the teacher and student supergraphs are aligned automatically; training requires the two label sets to match exactly and falls back to their intersection with a warning if they do not.

3.4 Supernode edge weights

Grouping the neurons produces a simplified supergraph, whose edges we compute with the supernode adjacency of Ameisen et al. [2025],

$$\text{supernode_adjacency}(t, s) = \frac{\sum_{n_t \in N_t} \text{frac_external}(n_t) \cdot \sum_{n_s \in N_s} A'_{n_t, n_s}}{\sum_{n_t \in N_t} \text{frac_external}(n_t)}, \quad (5)$$

where N_t and N_s are the sets of nodes in supernodes t and s , and n_t, n_s refer to a node in the corresponding supernode. A' is an input-normalised graph adjacency matrix, in which the weights of

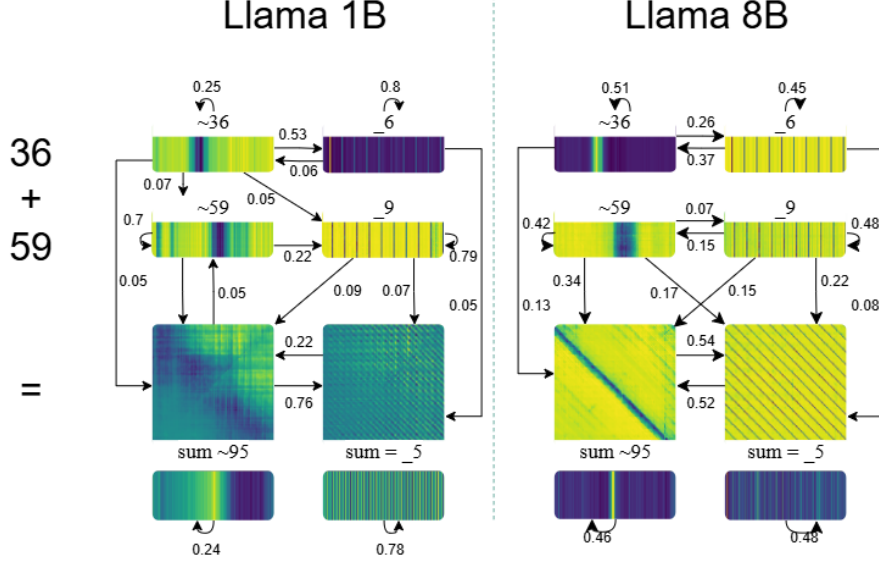


Figure 1: Supergraphs for the prompt “36+59=” in Llama-3.2-1B (left) and Llama-3.1-8B (right). Each node is a supernode from Section 3.3, drawn as the activation grid T_i of one representative member neuron — its activation across all 10^4 two-digit addition prompts, with operand a on one axis and b on the other — so that membership and computational role can be read off the same picture. Nodes ~ 36 and ~ 59 are arg1/arg2 range, $_6$ and $_9$ are arg1/arg2 units, $\text{sum} \sim 95$ and $\text{sum} = _5$ are sum range and sum units, and the bottom pair are the corresponding d1a supernodes. Edge labels are the supernode weights of (5); self-loops are within-supernode recurrence. The 8B grids are visibly cleaner: its $\text{sum} \sim 95$ node is a sharp anti-diagonal ($a + b \approx 95$) and its $\text{sum} = _5$ node a clean periodic lattice ($a + b \equiv 5 \pmod{10}$), whereas the 1B counterparts are diffuse and noisy — the same structure, represented less separably in the smaller model.

the edges targeting each node are divided by a constant so as to sum to 1; A'_{n_t, n_s} is the normalised weight of the edge into n_t from n_s . That normalisation runs over *all* nodes — neurons, token embeddings and logits alike, which are kept in the graph precisely so the denominator reflects every path by which a neuron can be influenced, even though they are dropped from the supergraph. Finally, $\text{frac_external}(n_t)$ is the fraction of the absolute-valued input weights to n_t that originate from nodes outside the supernode,

$$\text{frac_external}(n_t) = \frac{\sum_{j \notin N_t} |A_{n_t, j}|}{\sum_j |A_{n_t, j}|} = 1 - \sum_{n' \in N_t} |A'_{n_t, n'}|. \quad (6)$$

Weighting by frac_external is what turns (5) into an average rather than a sum: a member drawing most of its input from inside its own supernode counts for little, so a supernode whose members mainly feed one another cannot dominate its own inbound profile. Because A' normalises over all nodes while the supergraph spans only supernode columns, the rows of supernode_adjacency do not sum to one — which is why the routing profiles of Section 3.1 renormalise them.

3.5 Adapting the pipeline to Llama

The published methodology assumes a transcoder replacement model and a TransformerLens-style hook API [Ameisen et al., 2025]. Running it inside a Hugging Face training loop on Llama required several changes, most forced by the requirement that the student’s graph carry gradients back into θ_S .

Gated MLPs. Llama’s MLP is a SwiGLU block [Shazeer, 2020] with no single input matrix. With residual-stream input h at the node’s position,

$$a_i = \sigma(g_i) u_i, \quad v_i^{\text{in}} = \frac{\partial a_i}{\partial h} = \sigma(g_i) W_{\text{up}}[n_i, :] + u_i \sigma'(g_i) W_{\text{gate}}[n_i, :], \quad (7)$$

where $g_i = \langle W_{\text{gate}}[n_i, :], h \rangle$, $u_i = \langle W_{\text{up}}[n_i, :], h \rangle$ and σ is SiLU. The read direction is therefore a product-rule sum of two terms that itself depends on the activation, rather than a fixed row of W_{in} .

Differentiability. The pre-selection top- k is detached (indices are not differentiable), but activations and read/write vectors stay in the ambient autograd context, so gradients flow $\mathcal{L}_{\text{graph}} \rightarrow \text{supernode_adjacency} \rightarrow A' \rightarrow A \rightarrow v^{\text{out}} \rightarrow W_{\text{down}}$ back into the model. Supernode *membership* is recomputed each step but treated as fixed structure; only the aggregation (5) is rebuilt differentially, out-of-place, so that autograd views survive.

Shared attribution targets. Both models are attributed to the *same* logit token set — the teacher’s — and the student’s d1a supernodes are selected against the teacher’s logits. Otherwise the two graphs would be built over different output nodes, and would be non-stationary early in training when the student’s own output distribution is still wrong.

Known deviation. The published method freezes attention patterns and normalisation denominators when linearising, so edges reflect only the direct path. Our gradients pass through the full Hugging Face graph, so attention-softmax and RMSNorm Jacobians are included in (3), making our edges slightly less “direct-effect” than theirs. We have not measured the size of the difference. **es: Honest caveat — worth an ablation if we have time.**

3.6 Performance optimisations

An attribution graph costs one batch-expanded forward pass plus a vector–Jacobian product per node, which makes $\mathcal{L}_{\text{graph}}$ far more expensive per prompt than $\mathcal{L}_{\text{KD}}$. Four optimisations bring it inside a training loop. *Probe-set caching:* the residual-stream input to every MLP over all 10^4 prompts is computed once per model and cached to disk, keyed by model and dataset, then read back layer-by-layer, so every activation grid is recovered from the cache rather than by re-running the model. *Analytic pre-selection:* write-vector norms factor as $\|v_i^{\text{out}}\|_2 = |a_i| \|W_{\text{down}}[:, n_i]\|_2$, so the top- k runs in $O(n_{\text{pos}} d_{\text{mlp}})$ space without materialising the $[n_{\text{pos}}, d_{\text{mlp}}, d]$ tensor of write vectors. *Batching:* one batch-expanded forward pass serves a whole batch of target nodes, and all masks and activation grids are scored against each other in a single matmul for (4). *Bounded memory:* only n_{graph} prompts per batch get a graph, and each prompt’s graph loss is backpropagated and freed before the next graph is built, so peak memory is set by one prompt’s attribution graph rather than the batch’s.

4 Experimental Setup

[Experimental setup. Teacher/student pair and checkpoints; training data and splits; optimiser; schedule, batch size, n_{graph} ; which supernode categories are enabled; baselines (SFT, standard KD, KD + graph loss); evaluation metric and prompt format; hardware and wall-clock cost per step.]

5 Results

[Results. Accuracy against the baselines; graph loss and KD loss curves; whether supernode routing actually converges toward the teacher’s; per-category breakdown of which supernodes align and which do not.]

6 Analysis

[Analysis. Qualitative before/after supergraphs for the student; do the student’s activation grids sharpen over training? Ablations: λ , KL vs. other divergences, number of supernodes, n_{per} , and the frozen-attention deviation of Section 3.5.]

7 Limitations

[Limitations. Single task family (two-digit addition) and a single teacher/student pair; attribution graphs are built per prompt and only at sampled positions; edge sign is discarded by the input normalisation; supernode membership is recomputed but not itself differentiable.]

8 Conclusion

[*Conclusion.*]

References

- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *Advances in Neural Information Processing Systems*, 2024.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.
- Neel Nanda. Attribution patching: Activation patching at industrial scale. <https://www.neelnanda.io/mechanistic-interpretability/attribution-patching>, 2023.
- Noam Shazeer. GLU variants improve transformer. In *arXiv preprint arXiv:2002.05202*, 2020.

A Additional Details

[*Appendix. Full hyperparameters; the complete ANOVA category list including the composite and combination-sum categories omitted from Section 3.3; additional supergraph figures; per-supernode activation-heatmap montages.*]